Linear Regression with Multiple Variables

Comprehensive Cheatsheet

Machine Intelligence - Andrés Masegosa - Fall 2025

Contents

1 From Single to Multiple Features **2**

1.1 Motivation: Why Multiple Features? 2

1.2 Example Dataset 2

2 Notation for Multiple Features **3**

3 Multiple Linear Regression Model **5**

3.1 The Model Formula 5

3.2 Comparison with Single Variable 7

4 Cost Function for Multiple Features **7**

5 Gradient Descent for Multiple Features **9**

5.1 The Algorithm 9

5.2 Computing the Gradients 10

5.3 Complete Algorithm 10

6 Vectorization **11**

6.1 Why Vectorization? 11

6.2 Vectorized Gradient Descent 12

7 Feature Scaling **14**

7.1 The Problem 14

7.2 Feature Scaling Methods 14

8 Feature Engineering **16**

8.1 Creating New Features 16

9 Polynomial Regression **18**

9.1 Beyond Linear Relationships 18

9.2 Alternative: Other Basis Functions 20

10 The Problem of Overfitting **21**

10.1 Understanding Overfitting 21

10.2 Detecting Overfitting 22

10.3 Addressing Overfitting 24

11 Key Formulas Summary **25**

12 Summary **27**

1 From Single to Multiple Features

1.1 Motivation: Why Multiple Features?

The Real-World Problem

Previously: We predicted house price using only one feature (house size)
Reality: House prices depend on many factors!

- Size of the house (in square feet)
- Number of bedrooms
- Number of floors
- Age of the home (in years)
- Location, condition, amenities, etc.

The Question: How do we extend linear regression to handle multiple input features?
The Answer: Multiple Linear Regression (also called Multivariate Linear Regression)

1.2 Example Dataset

House Price Dataset with Multiple Features

Sample Data:

Size (ft^2)	Bedrooms	Floors	Age (years)	Price ($1000's)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178

Notice: Now we have 4 input features instead of just 1!
Each row is still one training example, but now each example has multiple features.

2 Notation for Multiple Features

Extended Notation - MEMORIZE THIS

New Symbols:

- n = **number of features**
  - In our house example: $n = 4$ (size, bedrooms, floors, age)
  - Previously we had $n = 1$ (only size)
- $\vec{x}$ = **feature vector** (vector of all input features)
  - Contains all n features: $\vec{x} = [x_1, x_2, x_3, \ldots, x_n]$
  - This is a column vector in $\mathbb{R}^n$
- x_j = **j-th feature**
  - x_1 = first feature (e.g., size)
  - x_2 = second feature (e.g., bedrooms)
  - x_j = j-th feature
- $\vec{x}^{(i)}$ = **feature vector for i-th training example**
  - Contains all features for example i
  - $\vec{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)}]$
- $x_j^{(i)}$ = **value of feature j in i-th training example**
  - Example: $x_2^{(3)}$ = number of bedrooms in the 3rd house
  - First superscript (i) = which training example
  - Subscript j = which feature

Important: The arrow notation $\vec{x}$ indicates a vector (multiple values), while x was a scalar (single value).

Using the New Notation

From our example table:
First training example ($i = 1$):

- $\vec{x}^{(1)} = [2104, 5, 1, 45]$
- $x_1^{(1)} = 2104$ (size in ft^2)
- $x_2^{(1)} = 5$ (bedrooms)
- $x_3^{(1)} = 1$ (floors)
- $x_4^{(1)} = 45$ (age)
- $y^{(1)} = 460$ (price)

Fourth training example ($i = 4$):

- $\vec{x}^{(4)} = [852, 2, 1, 36]$
- $x_1^{(4)} = 852$ (size)
- $x_2^{(4)} = 2$ (bedrooms)
- $y^{(4)} = 178$ (price)

Number of features: $n = 4$

3 Multiple Linear Regression Model

3.1 The Model Formula

Multiple Linear Regression Model

Extended Formula:

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n + b$$

Compact Notation using Vectors:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

where:

- $\vec{w} = [w_1, w_2, \ldots, w_n]$ = **weight vector** (parameters for each feature)
- $\vec{x} = [x_1, x_2, \ldots, x_n]$ = **feature vector** (input)
- $\vec{w} \cdot \vec{x}$ = **dot product** $= w_1x_1 + w_2x_2 + \cdots + w_nx_n$
- b = **bias term** (y-intercept)

Plain English:

- Each feature x_j has its own weight w_j
- We multiply each feature by its weight
- Add all these products together
- Add the bias b
- Result is our prediction

Understanding the Dot Product

What is $\vec{w} \cdot \vec{x}$?
The dot product is:

$$\vec{w} \cdot \vec{x} = w_1x_1 + w_2x_2 + \cdots + w_nx_n = \sum_{j=1}^{n} w_jx_j$$

In Python/NumPy:

```
prediction = np.dot(w, x) + b
# or equivalently
prediction = w @ x + b
```

Plain English: Multiply corresponding elements and sum them up. This is much faster than writing loops!

Concrete Example

Suppose we learned these parameters:

- $w_1 = 0.1$ (weight for size)
- $w_2 = 4$ (weight for bedrooms)
- $w_3 = 10$ (weight for floors)
- $w_4 = -2$ (weight for age - negative because older = cheaper)
- $b = 80$ (base price)

Predict price for a house:

- Size = 2000 ft², Bedrooms = 3, Floors = 2, Age = 20
- $\vec{x} = [2000, 3, 2, 20]$

Calculation:

$$\begin{aligned} f_{\vec{w},b}(\vec{x}) &= w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ &= 0.1(2000) + 4(3) + 10(2) + (-2)(20) + 80 \\ &= 200 + 12 + 20 - 40 + 80 \\ &= 272 \end{aligned}$$

Prediction: \$272,000
Interpretation of weights:

- $w_1 = 0.1$: Each extra square foot adds \$100 to price
- $w_2 = 4$: Each extra bedroom adds \$4,000
- $w_3 = 10$: Each extra floor adds \$10,000
- $w_4 = -2$: Each extra year of age subtracts \$2,000

3.2 Comparison with Single Variable

Single vs Multiple Variables

Single Variable Linear Regression ($n = 1$):

$$f_{w,b}(x) = wx + b$$

- One weight w
- One feature x
- Line in 2D space

Multiple Variable Linear Regression ($n > 1$):

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

- Multiple weights $w_1, w_2, \ldots, w_n$
- Multiple features $x_1, x_2, \ldots, x_n$
- Hyperplane in $(n+1)$-dimensional space

Key Insight: Multiple linear regression is just a natural extension! The concepts remain the same.

4 Cost Function for Multiple Features

Cost Function - Squared Error

Formula (exactly the same structure as before!):

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2$$

Expanded Version:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)} \right)^2$$

Even more expanded:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(w_1x_1^{(i)} + w_2x_2^{(i)} + \cdots + w_nx_n^{(i)} + b - y^{(i)} \right)^2$$

What Changed?

- The model f now depends on $\vec{w}$ (vector) instead of w (scalar)
- The input $\vec{x}^{(i)}$ is now a vector instead of scalar
- The cost function structure is EXACTLY the same!

Plain English: For each training example, compute the prediction error, square it, sum over all examples, and take the average. Nothing fundamentally changed!

Understanding the Cost Function

The Goal: Find the best values of $\vec{w}$ and b that minimize $J(\vec{w}, b)$
What we're minimizing:

$$\min_{\vec{w},b} J(\vec{w}, b)$$

This means finding:

- $w_1, w_2, \ldots, w_n$ (one weight for each feature)
- b (the bias term)

Visualization Challenge:

- For $n = 1$: Cost function $J(w, b)$ is a 3D surface (bowl shape)
- For $n = 2$: Cost function $J(w_1, w_2, b)$ is in 4D (hard to visualize)
- For $n > 2$: Cost function is in $(n + 2)$ dimensions (impossible to visualize)

Good News: The math and algorithms work the same way regardless of dimensions!

5 Gradient Descent for Multiple Features

5.1 The Algorithm

Gradient Descent - Multiple Features

Algorithm:

$$
\begin{aligned}
&\text{repeat until convergence}\{ \\
&\qquad w_1 = w_1 - \alpha \frac{\partial}{\partial w_1} J(\vec{w}, b) \\
&\qquad w_2 = w_2 - \alpha \frac{\partial}{\partial w_2} J(\vec{w}, b) \\
&\qquad \vdots \\
&\qquad w_n = w_n - \alpha \frac{\partial}{\partial w_n} J(\vec{w}, b) \\
&\qquad b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \\
&\qquad \}
\end{aligned}
$$

Compact Notation:

$$
\begin{aligned}
&\text{repeat until convergence}\{ \\
&\qquad w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \quad \text{for } j = 1, 2, \ldots, n \\
&\qquad b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \\
&\qquad \}
\end{aligned}
$$

Plain English:

- Update each weight w_j based on its gradient
- Also update bias b
- All updates happen simultaneously
- Repeat until convergence

5.2 Computing the Gradients

Gradient Formulas

For each weight w_j:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

For bias b:

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)$$

Pattern Recognition:

- Gradient for w_j includes the term $x_j^{(i)}$ (the j-th feature)
- Gradient for b has no x term
- Both include the prediction error: $f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}$
- Both average over all m training examples

Plain English: The gradient tells us how much to adjust each parameter. It depends on:

1. How wrong our predictions are (the error term)
2. The corresponding feature value (for weights)
3. Averaged across all examples

5.3 Complete Algorithm

Full Gradient Descent for Multiple Linear Regression

Initialize: $w_1 = 0, w_2 = 0, \ldots, w_n = 0, b = 0$ (or random values)
Repeat until convergence:

$$\text{For } j = 1 \text{ to } n:$$

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)$$

where $f_{\vec{w},b}(\vec{x}^{(i)}) = \vec{w} \cdot \vec{x}^{(i)} + b$
Important Notes:

- Compute ALL gradients first using OLD values of $\vec{w}$ and b
- Then update ALL parameters simultaneously
- This is called "simultaneous update"

Simultaneous Update - CRITICAL

WRONG (sequential update):

```
w1 = w1 - alpha * gradient_w1
w2 = w2 - alpha * gradient_w2  # Uses NEW w1!
b = b - alpha * gradient_b      # Uses NEW w1, w2!
```

RIGHT (simultaneous update):

```
temp_w1 = w1 - alpha * gradient_w1
temp_w2 = w2 - alpha * gradient_w2  # Uses OLD w1
temp_b = b - alpha * gradient_b      # Uses OLD w1, w2

w1 = temp_w1
w2 = temp_w2
b = temp_b
```

Why It Matters: Simultaneous update ensures consistent gradient computation across all parameters.

6 Vectorization

6.1 Why Vectorization?

The Performance Problem

Loop-Based Implementation (slow):

```
prediction = b
for j in range(n):
    prediction += w[j] * x[j]
```

Vectorized Implementation (fast):

```
prediction = np.dot(w, x) + b
```

Speed Comparison:

- For $n = 1000$ features: Vectorized is 10-100x faster
- For $n = 1,000,000$ features: Vectorized is 100-1000x faster
- Vectorization uses optimized hardware (CPU/GPU parallelism)

Plain English: Instead of computing one feature at a time in a loop, vectorization computes all features at once using optimized low-level code. This is essential for modern machine learning!

6.2 Vectorized Gradient Descent

Vectorized Implementation

Compute Predictions for All Examples (vectorized):

```
# X is (m, n) matrix: m examples, n features
# w is (n,) vector
# b is scalar
predictions = X @ w + b  # Shape: (m,)
```

Compute All Errors:

```
errors = predictions - y  # Shape: (m,)
```

Compute All Gradients:

```
gradient_w = (1/m) * (X.T @ errors)  # Shape: (n,)
gradient_b = (1/m) * np.sum(errors)  # Scalar
```

Update Parameters:

```
w = w - alpha * gradient_w
b = b - alpha * gradient_b
```

Key Insight: All loops are gone! Everything is matrix/vector operations.

Understanding the Matrix Notation

Data Matrix X:

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \cdots & x_n^{(m)} \end{bmatrix}$$

- Shape: $(m \times n)$
- Each row is one training example
- Each column is one feature

Weight Vector $\vec{w}$:

$$\vec{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

- Shape: $(n \times 1)$ or $(n,)$
- One weight per feature

Matrix Multiplication $X\vec{w}$:

- Result shape: $(m \times 1)$ or $(m,)$
- Computes predictions for all examples at once!
- i-th element $= \vec{w} \cdot \vec{x}^{(i)}$

7 Feature Scaling

7.1 The Problem

Why Feature Scaling Matters

Problem: Features have very different ranges!
Example:

- x_1 (size): ranges from 300 to 2000 (large numbers)
- x_2 (bedrooms): ranges from 1 to 5 (small numbers)
- x_3 (age): ranges from 0 to 100 (medium numbers)

Consequence:

- Cost function becomes very elongated (elliptical bowl)
- Gradient descent takes tiny steps in some directions, huge steps in others
- Convergence is VERY slow
- Learning rate is hard to tune

Solution: Scale all features to similar ranges!

7.2 Feature Scaling Methods

Method 1: Divide by Maximum

Formula:

$$x_j^{(i)} := \frac{x_j^{(i)}}{\max(x_j)}$$

Result: All features in range $[0, 1]$
Example:

- Original size: 300 to 2000
- Scaled size: $\frac{300}{2000}$ to $\frac{2000}{2000} = 0.15$ to 1.0

Simple but crude: Affected by outliers

Method 2: Mean Normalization

Formula:

$$x_j^{(i)} := \frac{x_j^{(i)} - \mu_j}{\max(x_j) - \min(x_j)}$$

where μ_j = mean of feature j
Result: Features roughly in range $[-1, 1]$ with mean 0
Example:

- Size: mean = 1200, min = 300, max = 2000
- Scaled: $\frac{x_1 - 1200}{2000 - 300} = \frac{x_1 - 1200}{1700}$

Better: Centers data around zero

Method 3: Z-score Normalization (Standardization)

Formula:

$$x_j^{(i)} := \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

where:

- μ_j = mean of feature j
- σ_j = standard deviation of feature j

Result: Features have mean 0 and standard deviation 1
Example:

- Size: mean = 1200, std = 400
- Scaled: $\frac{x_1 - 1200}{400}$

Most Common: Works well in practice, handles outliers better

Feature Scaling in Python

```
import numpy as np

# Method 1: Divide by max
X_scaled = X / np.max(X, axis=0)

# Method 2: Mean normalization
mu = np.mean(X, axis=0)
X_scaled = (X - mu) / (np.max(X, axis=0) - np.min(X, axis=0))

# Method 3: Z-score normalization (standardization)
mu = np.mean(X, axis=0)
sigma = np.std(X, axis=0)
X_scaled = (X - mu) / sigma

# Using scikit-learn (recommended)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

When to Use Feature Scaling

Always scale when:

- Features have very different ranges
- Using gradient descent
- Using algorithms sensitive to scale (neural networks, SVM, k-NN)

May not need scaling for:

- Decision trees and random forests
- Features already on similar scales

Important: Apply the same scaling to test data using parameters (mean, std) from training data!

8 Feature Engineering

8.1 Creating New Features

What is Feature Engineering?

Definition: Using intuition and domain knowledge to create new features by transforming or combining existing features.
Goal: Help the model capture patterns that aren't obvious with original features alone.
Example: For house prices:

- Original features: frontage (x_1), depth (x_2)
- Engineered feature: area = frontage $\times$ depth ($x_3 = x_1 \times x_2$)
- Why? Area might be more predictive than dimensions separately!

Feature Engineering Examples

Example 1: Size per Bedroom

- Original: x_1 = size, x_2 = bedrooms
- New feature: $x_3 = \frac{x_1}{x_2}$ = size per bedroom
- Why? Captures room spaciousness

Example 2: Age Categories

- Original: x_1 = age (continuous)
- New features: is_new (age < 5), is_old (age > 50)
- Why? Non-linear effect of age

Example 3: Interaction Terms

- Original: x_1 = size, x_2 = quality_rating
- New feature: $x_3 = x_1 \times x_2$
- Why? Large size matters more if quality is high

Model with engineered features:

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

where $x_3 = x_1 \times x_2$

Feature Engineering Best Practices

Do:

- Use domain knowledge
- Try mathematical transformations (products, ratios, logs)
- Create features that capture your intuition
- Test if new features improve performance

Don't:

- Add too many features blindly (overfitting risk)
- Forget to scale new features
- Ignore whether features make logical sense

Trade-off: More features = more expressive model, but also more risk of overfitting

9 Polynomial Regression

9.1 Beyond Linear Relationships

The Limitation of Linear Models

Problem: What if the relationship isn't a straight line?
Example: House price vs size might follow a curve:

- Small houses: price increases quickly with size
- Large houses: price increases slowly with size
- This is a curved (non-linear) relationship!

Solution: Use polynomial features to fit curves while still using linear regression!

Polynomial Regression

Idea: Create new features that are powers of existing features
Quadratic Model (degree 2):

$$f_{\vec{w},b}(x) = w_1 x + w_2 x^2 + b$$

Cubic Model (degree 3):

$$f_{\vec{w},b}(x) = w_1 x + w_2 x^2 + w_3 x^3 + b$$

General Polynomial (degree d):

$$f_{\vec{w},b}(x) = w_1 x + w_2 x^2 + \cdots + w_d x^d + b$$

Key Insight: This is still linear regression!

- Set $x_1 = x$, $x_2 = x^2$, $x_3 = x^3$, etc.
- Now it's: $f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$
- Same algorithm, just different features!

Polynomial Regression in Practice

Original feature: x = house size
Create polynomial features:

- $x_1 = x$ (size)
- $x_2 = x^2$ (size squared)
- $x_3 = x^3$ (size cubed)

Model:

$$f_{\vec{w},b}(x) = w_1 x + w_2 x^2 + w_3 x^3 + b$$

Example with numbers:

- House size: $x = 10$ (thousands of sq ft)
- Features: $x_1 = 10$, $x_2 = 100$, $x_3 = 1000$
- Notice: Features have VERY different scales!

Critical: You MUST scale polynomial features!

Choosing Polynomial Degree

Degree too low (underfitting):

- Model too simple
- Can't capture true pattern
- High training error
- High test error

Degree just right:

- Captures true pattern
- Good training error
- Good test error

Degree too high (overfitting):

- Model too complex
- Fits training data perfectly (even noise)
- Very low training error
- Very high test error

Rule of thumb: Start simple (degree 2), then increase if needed. Use validation set to choose!

Fitting Different Polynomials

Data: House prices that follow a curved pattern
Linear fit ($w_1x + b$):

- Straight line through data
- Misses the curve
- Underfits

Quadratic fit ($w_1x + w_2x^2 + b$):

- Smooth curve
- Fits data well
- Just right!

High-degree fit ($w_1x + \cdots + w_{10}x^{10} + b$):

- Wiggly curve passing through all points
- Fits training data perfectly
- Overfits (poor generalization)

9.2 Alternative: Other Basis Functions

Beyond Polynomials

General Idea: Use any transformation of features!
Square Root Model:

$$f_{\vec{w},b}(x) = w_1x + w_2\sqrt{x} + b$$

Logarithmic Model:

$$f_{\vec{w},b}(x) = w_1x + w_2\log(x) + b$$

Mixed Model:

$$f_{\vec{w},b}(x) = w_1x + w_2x^2 + w_3\sqrt{x} + b$$

Why? Different functions capture different types of patterns:

- Polynomials: Can go to $\pm\infty$
- Square root: Flattens out for large x
- Log: Grows slowly, good for diminishing returns

Plain English: You're not limited to polynomials! Use whatever transformations make sense for your problem.

10 The Problem of Overfitting

10.1 Understanding Overfitting

Three Scenarios

1. **Underfitting (High Bias)**:
   - Model is too simple
   - Doesn't capture the pattern in data
   - Poor performance on training data
   - Poor performance on new data
   - Example: Linear model for curved data
2. **Just Right (Good Fit)**:
   - Model complexity matches data complexity
   - Captures the true pattern
   - Good performance on training data
   - Good performance on new data
   - This is what we want!
3. **Overfitting (High Variance)**:
   - Model is too complex
   - Fits training data too well (including noise)
   - Excellent performance on training data
   - Poor performance on new data
   - Example: High-degree polynomial

The Central Challenge of Machine Learning

Bias-Variance Tradeoff:

- Simple models: High bias, low variance (underfit)
- Complex models: Low bias, high variance (overfit)
- Goal: Find the sweet spot in the middle

Overfitting happens when:

- Too many features relative to training examples
- Features are too complex (high-degree polynomials)
- Model memorizes training data instead of learning patterns

Quote: "All things being equal, the simplest solution tends to be the best one." - William of Ockham (Occam's Razor)

10.2 Detecting Overfitting

How to Detect Overfitting

Split Data:

- **Training Set** (70-80%): Used to train model
- **Validation/Test Set** (20-30%): Used to evaluate model

Compute Two Errors:

- **Training Error**: $J(\vec{w}, b)$ on training data
- **Test Error**: $J(\vec{w}, b)$ on test data

Diagnose:

Scenario	Training Error	Test Error
Underfitting	High	High
Good Fit	Low	Low
Overfitting	Very Low	High

Key Signal: Large gap between training and test error indicates overfitting!

Example: Polynomial Degree vs Error

Experiment: Try different polynomial degrees

Degree	Training Error	Test Error
1	25.0	26.0
Underfitting		
2	10.0	11.0
Good		
3	8.0	9.5
Good		
5	5.0	15.0
Overfitting		
10	0.1	50.0
Severe overfitting		

Observation:

- As degree increases, training error decreases
- Test error first decreases, then increases
- Large gap signals overfitting

Best choice: Degree 2 or 3 (lowest test error)

10.3 Addressing Overfitting

Solutions to Overfitting

1. **Get More Training Data**:
   - More examples help the model generalize
   - Most effective but not always possible
2. **Reduce Number of Features**:
   - Manually select important features
   - Use feature selection algorithms
   - Trade-off: Might lose useful information
3. **Regularization**:
   - Keep all features but reduce their impact
   - Add penalty for large weights
   - Next lecture topic!
4. **Cross-Validation**:
   - Use multiple train/test splits
   - Get more reliable estimates of test error
   - Helps choose model complexity

11 Key Formulas Summary

Everything You Need to Memorize

1. **Multiple Linear Regression Model**:

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \vec{w} \cdot \vec{x} + b$$

2. **Cost Function**:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2$$

3. **Gradient Descent**:

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \quad \text{for } j = 1, \ldots, n$$
$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

4. **Gradients**:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$
$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)$$

5. **Feature Scaling (Z-score)**:

$$x_j^{(i)} := \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

6. **Polynomial Regression**:

$$f_{\vec{w},b}(x) = w_1x + w_2x^2 + \cdots + w_dx^d + b$$

12 Summary

Essential Points

1. **Multiple Features**:
   - Extend linear regression to handle n features
   - Each feature has its own weight w_j
   - Model: $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
2. **Gradient Descent**:
   - Update each weight based on its gradient
   - Simultaneous update of all parameters
   - Same algorithm structure as single variable
3. **Vectorization**:
   - Use matrix operations instead of loops
   - 10-1000x faster for large datasets
   - Essential for modern machine learning
4. **Feature Scaling**:
   - Normalize features to similar ranges
   - Makes gradient descent converge much faster
   - Use z-score normalization: $\frac{x-\mu}{\sigma}$
5. **Feature Engineering**:
   - Create new features from existing ones
   - Use domain knowledge and intuition
   - Examples: products, ratios, transformations
6. **Polynomial Regression**:
   - Fit curves using powers of features
   - Still linear regression (linear in parameters)
   - Must scale polynomial features!
7. **Overfitting**:
   - Model too complex: fits training data too well
   - Symptoms: Low training error, high test error
   - Solutions: More data, fewer features, regularization
8. **Model Selection**:
   - Split data: training vs test sets
   - Evaluate on test set to detect overfitting
   - Choose model with best test performance

Looking Ahead

What's Next?

- **Regularization**: Prevent overfitting by penalizing large weights
- **Classification**: Logistic regression for yes/no predictions
- **Neural Networks**: Extend linear models with non-linear activations
- **Advanced Optimization**: Faster algorithms than basic gradient descent

The Foundation: Multiple linear regression is the building block for more complex models. Master this first!